

MÔ TẢ ĐỒ ÁN MÔN HỌC IT003.Q21.CTTN

THÔNG TIN SINH VIÊN THỰC HIỆN

Mã sinh viên: 25521787

Họ và tên: Võ Quốc Thịnh

TÊN ĐỀ TÀI

Game Pacman cơ bản

NỘI DUNG VÀ NHIỆM VỤ/YÊU CẦU (Liệt kê dưới dạng gạch đầu dòng từng nội dung và yêu cầu, nhiệm vụ cần giải quyết)

1. Xây dựng bản đồ (Map & Environment)

- **Biểu diễn dữ liệu:** Sử dụng mảng 2 chiều (ma trận/grid) để mã hóa bản đồ. Ví dụ: 0 là đường đi, 1 là tường, 2 là thức ăn thường, 3 là thức ăn sức mạnh (power pellet).
- **Thiết kế Layout:** Tạo các khối tường (maze) có tính đối xứng hoặc cân bằng. Cần xây dựng "chuồng" (Ghost House) ở giữa bản đồ cho các nhân vật phản diện và các đường hầm (tunnels) ở hai mép màn hình để nhân vật có thể đi xuyên từ bên này sang bên kia.
- **Xử lý đồ họa (Render):** Chuyển đổi các giá trị từ ma trận thành các block đồ họa tương ứng (vẽ tường, rải các chấm tròn làm thức ăn).

2. Tạo ra nhân vật (Entities/Characters)

- **Pacman (Người chơi):**
 - Thiết lập vị trí sinh ra (spawn point) mặc định, tốc độ di chuyển và số mạng (lives).
 - Xử lý Input: Nhận tín hiệu điều khiển từ bàn phím (các phím mũi tên hoặc W, A, S, D) để thay đổi hướng đi.
 - Hoạt ảnh (Animation): Xử lý hiệu ứng há/ngậm miệng dựa trên hướng đang di chuyển.
- **Ghosts (Nhân vật phản diện):**
 - Thiết lập vị trí spawn trong Ghost House.
 - Quản lý các trạng thái hoạt động:
 - *Chase* (Săn đuổi)
 - *Scatter* (Phân tán về các góc)
 - *Frightened* (Sợ hãi/Chuyển màu xanh - kích hoạt khi Pacman ăn thức ăn sức mạnh)

- *Eaten* (Bị ăn - chỉ còn đôi mắt bay về chuồng để hồi sinh).

3. Xác định điều kiện thua và thắng (Game Logic & States)

- **Điều kiện Thắng:** Hệ thống liên tục kiểm tra số lượng thức ăn (dots) trên bản đồ. Khi số lượng bằng 0, game chuyển sang trạng thái Thắng (hoặc load cấp độ tiếp theo).
- **Điều kiện Thua:** Viết hàm xử lý va chạm (Collision Detection) giữa tọa độ của Pacman và Ghost. Nếu chạm nhau khi Ghost đang ở trạng thái bình thường -> Pacman mất 1 mạng, reset vị trí tất cả về điểm xuất phát. Nếu số mạng = 0 -> Game Over.
- **Hệ thống điểm (Scoring):** Cộng điểm tăng dần cho từng hành động: ăn hạt nhỏ, ăn hạt to, ăn trái cây thưởng, và chuỗi điểm liên hoàn khi ăn liên tiếp các Ghost đang sợ hãi.

4. Làm level dễ: Sử dụng random để Ghost di chuyển

- **Logic di chuyển ngẫu nhiên:** Khi Ghost đi đến một ngã ba hoặc ngã tư, thuật toán sẽ thu thập các hướng có thể đi (không bị chặn bởi tường).
- **Quy tắc:** Chọn ngẫu nhiên (Random) một trong các hướng đó để đi tiếp. Thường sẽ lập trình thêm điều kiện *không được quay ngược lại* hướng vừa đi để tránh tình trạng Ghost bị kẹt hoặc đi giạt lùi, giúp chuyển động trông tự nhiên hơn.

5. Làm level khó: Sử dụng các thuật toán tìm đường cho Ghost

- **Áp dụng thuật toán:** Đưa các kiến thức về cấu trúc dữ liệu và thuật toán đồ thị vào lưới ma trận. Bạn có thể dùng Breadth-First Search (BFS) để tìm đường đi ngắn nhất cơ bản.
- **Tạo đặc trưng riêng (Giống bản gốc):** Thay vì tất cả đều đuổi theo một ô, có thể chỉnh mục tiêu (Target Node) của từng con:
 - *Ghost Đỏ:* Target là ô Pacman đang đứng.
 - *Ghost Hồng:* Target là ô nằm trước mặt Pacman 4 bước (đón đầu).
 - *Ghost Cam:* Dùng BFS đuổi theo, nhưng nếu tiến vào quá gần (bán kính 8 ô) thì tự động quay đầu bỏ chạy về góc bản đồ.

6. Kết nối mọi thứ lại với nhau để hoàn thiện (Game Loop)

- **Vòng lặp Game (Main Loop):** Đảm bảo chương trình chạy liên tục theo một trình tự: Nhận Input -> Cập nhật Logic/Thuật toán (Update) -> Xử lý Va chạm -> Vẽ lại màn hình (Render) dựa trên chỉ số FPS cố định.
- **Quản lý màn hình (Scenes):** Tích hợp Menu chính (Start, Exit), màn hình hiển thị điểm số, và màn hình Game Over.

Chú ý, một số quy định cần biết khi thực hiện đồ án môn học

1. Tổ chức lưu trữ project trên git (public)
2. Có thể sử dụng các công cụ hỗ trợ để thiết kế giao diện, cơ sở dữ liệu. Phần liên quan đến CTDL & TT cần được sinh viên thực hiện toàn bộ.
3. Được phép sử dụng các thư viện, đoạn mã dựng sẵn; khi dùng cần chỉ rõ nguồn
4. Bắt buộc viết docstring